

Unifying Sequence-to-Function Models with Continuous Flow Matching with STREAM

We propose Sequence-conditioned Trajectories via Regulatory Element Matching (STREAM), a framework for learning continuous gene expression dynamics from time-resolved single-cell data. STREAM unifies sequence-to-function models with continuous flow matching, enabling the inference of gene regulatory dynamics across developmental time.

1 Background

By training to predict diverse chromatin marks, sequence-to-function models learn representations of regulatory DNA sequences, in particular candidate cis-regulatory elements (cCRE). The representation ideally summarizes sequence features relevant to gene regulation, such as transcription factor motifs and higher-order sequence grammar.

The state-of-the-art paradigm for learning ordinary differential equation (ODE) models parameterized by a continuous-time vector field is Conditional Flow Matching (CFM). The CFM “trick” is framing model training as a regression that fits target velocities along simple paths between data distributions at different time points. This avoids the high computational cost of forward simulating the ODE, enabling efficient, scalable training. For time-resolved single-cell data, this corresponds to fitting a vector field that transports cell-state distributions between observed time points.

2 Model setup

Continuous cell state is represented as $x_t \in \mathbb{R}^G$ at time t , with G being the number of genes in the selected modeling panel. For the mouse development implementation, G is a configurable protein-coding highly variable gene subset so that STREAM and the baseline CFM are compared on exactly the same output coordinates. We observe the expression profiles of cells at a discrete set of time points (e.g., embryonic days) $\mathcal{T} = \{t_0, t_1, \dots, t_K\}$. To make training comparable across datasets we require t is on the same scale (e.g. days). The empirical data distribution at time t_k is $p_k(x)$.

Let $\mathcal{C}_g$ be the set of cCREs associated with gene g . In the mouse development implementation, cCREs are linked to genes if their midpoint is within 100 kb of the gene transcription start site (TSS) in mm10. Let $z_c \in \mathbb{R}^d$ denote a precomputed representation of cCRE c , generated by AlphaGenome from local DNA sequence. The set of all precomputed cCRE embeddings for gene g is $\mathbf{Z}_g = \{z_c \mid c \in \mathcal{C}_g\}$.

For each gene, one token is distinguished as the promoter token. The promoter token is the linked cCRE closest to the TSS if that cCRE is within 1 kb of the TSS. If the closest linked cCRE is farther than 1 kb, a synthetic promoter cCRE centered on the TSS is inserted. The promoter token is placed first in the token sequence. This guarantees that every gene has an explicit promoter representation even if distal cCRE annotation is sparse near the TSS.

We establish a sequence and state dependent map f from each gene’s sequence-derived regulatory elements and the instantaneous global cell state to that gene’s velocity. The cCREs associated with a gene are treated as a sentence of tokens processed by a transformer. Token features include the AlphaGenome embedding, signed distance to TSS, a promoter indicator, and optional positional information. The implementation supports no positional encoding, learned binned distance embeddings, and rotary positional embeddings (RoPE) applied using signed TSS-relative positions. Cell state x_t conditions the regulatory token processing either through FiLM or through cross-attention. The scalar output for each gene is read from the final promoter-token representation, analogous to a CLS readout.

We define a global vector field,

$$\frac{dx_t}{dt} = \begin{bmatrix} f(x_t, \mathbf{Z}_1) \\ \vdots \\ f(x_t, \mathbf{Z}_G) \end{bmatrix} \in \mathbb{R}^G$$

This allows the model to capture *trans*-regulatory effects because each gene-specific velocity depends on the full cell state x_t , not only on that gene’s expression level.

Note that we do not condition on time t to allow generalization across different time-scales. Additionally, genes are represented as a set of regulatory elements, rather than a single token, to force the model to learn global regulatory grammar rather than memorizing gene-specific dynamics.

Conditioning variants. The FiLM STREAM variant maps x_t to feature-wise scale and shift parameters that modulate regulatory token states before promoter readout. The cross-attention STREAM variant maps x_t to a small set of context tokens; regulatory tokens attend to these context tokens before promoter readout. Both variants share the same cCRE-gene links, AlphaGenome embeddings, promoter-token convention, minibatch OT coupling, and selected gene panel.

2.1 Architecture of the gene velocity model

For each selected gene g , the implementation packs up to M regulatory tokens into a fixed tensor. Token m for gene g has an AlphaGenome embedding $z_{gm} \in \mathbb{R}^{3072}$, signed TSS-relative distance d_{gm} , a promoter indicator p_{gm} , and a mask a_{gm} indicating whether the token is real rather than padding. The promoter token is always placed at index $m = 0$. Let

$$e_{gm}^{(0)} = W_z z_{gm} + \phi_d(d_{gm}/100,000) + E_p[p_{gm}] + \phi_{\text{pos}}(d_{gm}),$$

where W_z projects AlphaGenome embeddings to the model width, ϕ_d is a small MLP for signed distance, E_p is a learned promoter/non-promoter embedding, and ϕ_{pos} is optional positional information. In the current code, ϕ_{pos} can be absent, a learned binned-distance embedding, or RoPE applied to token features using signed distance. Padding tokens are masked in all self-attention layers.

The gene regulatory sequence is encoded by a shared transformer,

$$H_g = \text{TransformerEnc}(e_{g1}^{(0)}, \dots, e_{gM}^{(0)}),$$

with the same parameters used for every gene. This weight sharing is important: the model must learn regulatory grammar that transfers across genes rather than assigning each gene a free velocity parameter. The default mouse development configuration uses model width 256, 8 attention heads, 4 transformer layers, dropout 0.1, and at most 32 CRE/promoter tokens per gene.

The cell state x_t is incorporated after CRE self-attention. This makes f both sequence dependent and state dependent. Two conditioning mechanisms are implemented:

FiLM conditioning. A cell-state MLP maps x_t to feature-wise scale and shift vectors,

$$(\gamma(x_t), \beta(x_t)) = h_{\text{FiLM}}(x_t), \quad \tilde{H}_{bgm} = H_{gm} \odot (1 + \gamma_b) + \beta_b,$$

for cell b , gene g , and token m . A layer normalization and scalar head are then applied only to the promoter token,

$$f_g(x_t, \mathbf{Z}_g) = w^\top \text{LN}(\tilde{H}_{bg0}) + b.$$

Cross-attention conditioning. A cell-state MLP maps x_t to a small set of context tokens $C_b \in \mathbb{R}^{L \times D}$, with $L = 8$ by default. Regulatory tokens query these context tokens,

$$\tilde{H}_{bg:} = H_{g:} + \text{MHA}(Q = H_{g:}, K = C_b, V = C_b),$$

and the same promoter-token scalar head gives $f_g(x_t, \mathbf{Z}_g)$. This variant allows each gene’s regulatory tokens to extract different information from the global cell state.

The resulting vector field is formed by evaluating the same gene-level module for all selected genes:

$$f_\theta(x_t, \{\mathbf{Z}_g\}_{g=1}^G) = (f_1(x_t, \mathbf{Z}_1), \dots, f_G(x_t, \mathbf{Z}_G)).$$

The implementation does not use a learned gene identity embedding by default. Gene specificity enters through the set of linked CRE/promoter tokens and their sequence-derived embeddings.

Expression-only baseline. The standard CFM baseline uses the same selected genes, same minibatch OT coupling, same interpolation path, and same target velocity, but replaces the sequence-conditioned gene module with an MLP $f_{\text{CFM}}(x_t) : \mathbb{R}^G \rightarrow \mathbb{R}^G$. This isolates the contribution of CRE/AlphaGenome conditioning from the flow matching training setup.

Sequential Flow Matching Objective. To fit f across multiple time points, we match it to a target marginal vector field $\hat{f}_t$. We define independent flow matching objectives between consecutive time points $[t_k, t_{k+1}]$. Let $x_k \sim p_k(x)$ and $x_{k+1} \sim p_{k+1}(x)$ be sampled from the empirical distributions. We construct a linear path mapping x_k to x_{k+1} . For $t \in [t_k, t_{k+1}]$, define the normalized time step $\tau = \frac{t - t_k}{t_{k+1} - t_k}$.

The interpolated state x_t is:

$$x_t(\tau) = (1 - \tau)x_k + \tau x_{k+1}$$

The target conditional vector field, representing the velocity required to move from x_k to x_{k+1} is,

$$\hat{f}_t = \frac{x_{k+1} - x_k}{t_{k+1} - t_k}$$

The unified Conditional Flow Matching (CFM) objective minimizes the expected squared difference between the sequence-conditioned network f and the target vector field $\hat{f}_t$ across all consecutive time intervals:

$$\mathcal{L}(f) = \sum_{k=0}^{K-1} \mathbb{E}_{t \sim \mathcal{U}[t_k, t_{k+1}], x_k, x_{k+1} \sim p_{data}(x_k, x_{k+1})} \left[\left\| f(x_t, \{\mathbf{Z}_g\}_{g=1}^G) - \hat{f}_t \right\|^2 \right]$$

While it is valid to use the trivial joint distribution $p_{data}(x_k, x_{k+1}) = p_k(x_k)p_{k+1}(x_{k+1})$, STREAM uses minibatch optimal transport to couple cells between neighboring time points. Each minibatch contains cells from exactly one interval (t_k, t_{k+1}) . A pairwise cost is computed between sampled cells at t_k and sampled cells at t_{k+1} , an entropic OT coupling is solved on the minibatch, and training pairs are sampled from the resulting coupling.

3 Mouse development implementation

The implementation targets the JAX mouse development AnnData files and mm10 references. Gene models are read from the Ensembl GRCm38 GTF, and cCREs are read from the mouse cCRE BED file. TSS coordinates are converted from 1-based GTF coordinates to 0-based genomic positions. For positive-strand genes the TSS is the gene start; for negative-strand genes it is the gene end.

AlphaGenome embeddings are computed once and cached. The implementation uses the local PyTorch AlphaGenome port, mouse organism index 1, and the 128 bp trunk embeddings. For each real or synthetic cCRE window, the 128 bp embeddings are pooled to one fixed vector. Training reads the cached CRE token tensor and does not call AlphaGenome online.

4 Baseline and evaluation

STREAM is compared against a standard expression-only CFM baseline. The baseline uses the same selected genes, same neighboring-time minibatch sampler, same minibatch OT coupling, same interpolation path, and same velocity target, but it does not receive cCRE or AlphaGenome inputs.

Evaluation is performed on held-out time points. Whole embryonic days are removed from training intervals. Training uses only adjacent intervals whose two endpoints are both training time points. Evaluation reports held-out-timepoint losses and rollout diagnostics on intervals adjacent to or involving held-out days when those endpoints are available. This directly tests temporal generalization rather than embryo-level generalization.